What is Python?

Python is an interpreted, object-oriented, high-level programming language.

Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance.

Python supports modules and packages, which encourages program modularity and code reuse.

Python is a general-purpose language, meaning it can be used to create a variety of different programs and isn't specialized for any specific problems.

First python Program

In [6]:
```
print('This is my first  Python program')
```

```
This is my first
 Python program
```

In [2]:
```
print("Hello World!!!")
```

```
Hello World!!!
```

In [3]:
```
print("My name is _____")
```

```
My name is _____
```

In [4]:
```
print("My roll no. is _____.")
```

```
My roll no. is _____.
```

Comments

In [5]:
```
#Comments can be used to explain Python code.
#Comments can be used to make the code more readable.
#Comments can be used to prevent execution when testing code
```

Multi line comment

In [6]:
```
"""
 Comments can be used to explain Python code.
 Comments can be used to make the code more readable.
 Comments can be used to prevent execution when testing code
"""
```

Out[6]:
```
'\n Comments can be used to explain Python code.\n Comments can be used to make the code more readable.\n Comments can be used to prevent execution when testing code\n'
```

Indentation

The number of spaces is up to you as a programmer, but it has to be at least one.

You have to use the same number of spaces in the same block of code, otherwise Python will give you an error:

In [7]:
```python
if 7 > 2:
    print("Seven is greater than two!")
```

```
Seven is greater than two!
```

Python Variables

In [8]:
```python
x = 5
y = "Hello, World!"
print(x)
print((type(x)))
```

```
x=y
<class 'int'>
```

String variables can be declared either by using single or double quotes.

Case sensitive A or a

In [17]:
```python
print(y)
print((type)(y))
```

```
Hello, World!
<class 'str'>
```

Keywords and Literals

Keywords are the reserved words in Python. We cannot use a keyword as a variable name, function name or any other identifier. They are used to define the syntax and structure of the Python language. In Python, keywords are case sensitive.

Generally, literals are a notation for representing a fixed value in source code. Literals are the raw data that are assigned to variables or constants while programming. In python, we have different types of literals such as string literals, numeric literals, boolean literals and a special literal None

In [4]:
```
help('keywords')
```

```
Here is a list of the Python keywords.  Enter any keyword to get more help.

False               break               for                 not
None                class               from                or
True                continue            global              pass
__peg_parser__      def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
```

In [5]:
```
help('pass')
```

```
The "pass" statement
********************

   pass_stmt ::= "pass"

"pass" is a null operation — when it is executed, nothing happens. It
is useful as a placeholder when a statement is required syntactically,
but no code needs to be executed, for example:

   def f(arg): pass    # a function that does nothing (yet)

   class C: pass       # a class with no methods (yet)
```

Operators

```
Arithmetic operators
Assignment operators
```

Arithmetic operators

In [10]:
```
x = 3
y = 7
```

In [11]:
```
x+y
```

Out[11]: 10

In [12]:
```
x*y
```

Out[12]: 21

In [13]:
```
x-y
```

Out[13]: -4

In [14]:
```
x/y
```

Out[14]: 0.42857142857142855

Many other arithmetic operators are there.

Assignment operators

Assignment operators are used to assign values to variables:

In [15]:
```
x = 7
print(x)
```

7

In [16]:
```
x = 5
x += 3 # this is same as x= x+3
print(x)
```

```
8
```

In [17]:
```
x = 9
x *= 3 # this is same as x = x*3
print(x)
```

```
27
```

Comparison operators

Comparison operators are used to compare two values.

In [18]:
```
x = 8
y = 4
print(x == y)
# returns False because 8 is not equal to 4
```

```
False
```

In [19]:
```
x = 9
y = 3
print(x != y)
# returns True because 5 is not equal to 3
```

```
True
```

In [20]:
```
x = 7
y = 11
print(x > y)
# returns True because 5 is greater than 3
```

```
False
```

In [21]:
```
x = 6
y = 2
print(x >= y)
# returns True because five is greater, or equal, to 2
```

```
True
```

In [22]:
```
x = 9
y = 3
print(x <= y)
# returns False because 5 is neither less than or equal to 3
```

```
False
```

Logical operators

Logical operators are used to combine conditional statements:

In [23]:
```
x = 6
print(x > 4 and x == 6)
```

```
True
```

In [24]:
```
x = 9
print(x > 1 or x != 9)
```

```
True
```

In [25]:
```
x = 5
print(not(x > 3 and x < 10))
# returns False because not is used to reverse the result
```

```
False
```

Identity operators

Identity operators are used to compare the objects

Operators 'is' and 'is not'

is : Returns true if both variables are the same object x is y

is not : Returns true if both variables are not the same obiect x is not v

In [26]:

```
x = ["potato", "tomato"]
y = ["potato", "tomato"]
z = x

print(z is x)
# returns True because z is the same object as x

print(x is y)
# returns False because x is not the same object as y, even if they have the same content

print(x == y)
# to demonstrate the difference betweeen "is" and "==": this comparison returns True because x is equal to y
```

```
True
False
True
```

In [27]:

```
x = [4, 8]
y = [4, 8]
z = x

print(x is not z)
# returns False because z is the same object as x

print(x is not y)
# returns True because x is not the same object as y, even if they have the same content

print(x != y)
# to demonstrate the difference betweeen "is not" and "!=": this comparison returns False because x is equal to y
```

```
False
True
False
```

Membership operators

Membership operators are used to test if a sequence is presented in an object:

In [28]:
```
x = ["apple", "potato"]
print("banana" in x)
# returns False because a sequence with the value "banana" is not in the list
```

```
False
```

In [29]:
```
x = ["apple", "banana"]
print("pineapple" not in x)
# returns True because a sequence with the value "pineapple" is not in the list
```

```
True
```

membership operator

In [2]:
```
a=10
b=20
```

In [3]:
```
a and b
```

Out[3]:
```
20
```

In [4]:
```
a or b
```

Out[4]:
```
10
```

In [5]:
```
a & b
```

Out[5]:
```
0
```

In [9]:
```
not b
```

Out[9]: False

Data types in Python :

```
Numeric = 20, 20.5, 1j

String = "Hello", "World"

List = ["green", "blue", "red"]

Tuple = ("apple", "banana", "cherry")

Dictionary = {"green", "blue", "red"}
```

Python Numbers

There are three numeric types in Python:

- `int`
- `float`
- `complex`

Variables of numeric types are created when you assign a value to them:

• Int

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

In [30]:

```python
x = 1
y = 35656222554887711
z = -3255522

print(type(x))
print(type(y))
print(type(z))
```

```
<class 'int'>
<class 'int'>
<class 'int'>
```

• Float

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

In [31]:

```python
x = 1.10
y = 1.0
z = -35.59

print(type(x))
print(type(y))
print(type(z))
```

```
<class 'float'>
<class 'float'>
<class 'float'>
```

In [32]:

```python
x = 35e3
y = 12E4
z = -87.7e100

print(type(x))
print(type(y))
print(type(z))
```

```
<class 'float'>
<class 'float'>
<class 'float'>
```

• Complex

Complex numbers are written with a "j" as the imaginary part:

In [33]:

```
x = 3+5j
y = 5j
z = -5j

print(type(x))
print(type(y))
print(type(z))
```

```
<class 'complex'>
<class 'complex'>
<class 'complex'>
```

• **Type Conversion**

You can convert from one type to another with the int(), float(), and complex() methods:

In [18]:

```python
x = 3    # int
y = 7.3  # float
z = 7j   # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)

print(a)
print(b)
print(c)

print(type(a))
print(type(b))
print(type(c))
```

```
3.0
7
(3+0j)
<class 'float'>
<class 'int'>
<class 'complex'>
```

- **Random Number**

Python does not have a random() function to make a random number, but Python has a built-in module called random that can be used to make random numbers:

In [19]:

```python
#Import the random module, and display a random number between 5 and 20:
import random
print(random.randrange(5, 20))
```

```
10
```

Strings

• String Length To get the length of a string, use the len() function.

In [20]:
```
#The len() function returns the length of a string:
a = "Hello, World!"
print(len(a))
```

```
13
```

• Check String

To check if a certain phrase or character is present in a string, we can use the keyword in.

In [21]:
```
#Check if "free" is present in the following text:
txt = "The best things in life are free!"
print("free" in txt)
```

```
True
```

In [22]:
```
#Print only if "best" is present:
txt = "The best things in life are free!"
if "best" in txt:
  print("Yes, 'best' is present.")
```

```
Yes, 'best' is present.
```

• Check if NOT

To check if a certain phrase or character is NOT present in a string, we can use the keyword not in.

In [39]:
```
#Check if "expensive" is NOT present in the following text:
txt = "The best things in life are free!"
print("expensive" not in txt)
```

```
True
```

In [40]:
```
#print only if "expensive" is NOT present:
txt = "The best things in life are free!"
if "expensive" not in txt:
  print("Yes, 'expensive' is NOT present.")
```

```
Yes, 'expensive' is NOT present.
```

Boolean Values

In programming you often need to know if an expression is True or False. You can evaluate any expression in Python, and get one of two answers, True or False. When you compare two values, the expression is evaluated and Python returns the Boolean answer:

In [23]:
```
#The bool() function allows you to evaluate any value, and give you True or False in return,
x = 2==3
y = 1<9

print(bool(x))
print(bool(y))
```

```
False
True
```

List

Lists are used to store multiple items in a single variable. Lists are created using square brackets:

In [42]:
```
thislist = ["cat", "bat", "rat"]
print(thislist)
```

```
['cat', 'bat', 'rat']
```

List Items

List items are ordered, changeable, and allow duplicate values.

Ordered When we say that lists are ordered, it means that the items have a defined order, and that order will not change. If you add new items to a list, the new items will be placed at the end of the list.

Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

In [43]:
```
thislist = ["cat", "bat", "rat"]
thislist.append("watermelon")
print(thislist)
```

```
['cat', 'bat', 'rat', 'watermelon']
```

Allow Duplicates Since lists are indexed, lists can have items with the same value:

In [44]:
```
thislist = ["green", "blue", "black", "green", "pink"]
print(thislist)
```

```
['green', 'blue', 'black', 'green', 'pink']
```

List Length To determine how many items a list has, use the len() function:

In [45]:

```
#thislist = ["green", "pink", "blue"]
thislist = ["green", "blue", "black", "green", "pink"]
print(len(thislist))
```

```
5
```

List Items - Data Types

List items can be of any data type:

In [46]:

```
#String, int and boolean data types:
a = ["apple", "banana", "cherry"]
b = [1, 5, 7, 9, 3]
c = [True, False, False]

print((a))
print((b))
print((c))

print(type(a))
print(type(b))
print(type(c))
```

```
['apple', 'banana', 'cherry']
[1, 5, 7, 9, 3]
[True, False, False]
<class 'list'>
<class 'list'>
<class 'list'>
```

In [47]:

```
#A list with strings, integers and boolean values:
#A list can contain different data types:
list1 = ["abc", 45, False, 20, "female"]

print(list1)
```

```
['abc', 45, False, 20, 'female']
```

The list() Constructor

It is also possible to use the list() constructor when creating a new list.

In [48]:
```
#Using the list() constructor to make a List:
thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

```
['apple', 'banana', 'cherry']
```

Tuple

Tuples are used to store multiple items in a single variable. Tuples are written with round brackets.

In [49]:
```
#Create a Tuple:
thistuple = ("green", "blue", "white")
print(thistuple)
```

```
('green', 'blue', 'white')
```

Tuple Items

Tuple items are ordered, unchangeable, and allow duplicate values.

Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

In [50]:
```
#thistuple = ("cat", "bat", "rat")
#thistuple.append("watermelon")
#print(thistuple)
```

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value:

In [51]:
```
thistuple = ("green", "blue", "white", "green","white")
print(thistuple)
```

```
('green', 'blue', 'white', 'green', 'white')
```

Tuple Length

To determine how many items a tuple has, use the len() function:

In [52]:
```
thistuple = ("green", "blue", "white")
print(len(thistuple))
```

```
3
```

Create Tuple With One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

In [24]:
```
#One item tuple, remember the comma:
thistuple = ("green",)
print(type(thistuple))

#NOT a tuple
thistuple = ("green")
print(type(thistuple))
```

```
<class 'tuple'>
<class 'str'>
```

In [54]:

```python
#String, int and boolean data types:
tuple1 = ["green", "blue", "pink"]
tuple2 = [1, 5, 7, 9, 3]
tuple3 = [True, False, False]

print((tuple1))
print((tuple2))
print((tuple3))

print(type(tuple1))
print(type(tuple2))
print(type(tuple3))
```

```
['green', 'blue', 'pink']
[1, 5, 7, 9, 3]
[True, False, False]
<class 'list'>
<class 'list'>
<class 'list'>
```

Set

In [25]:

```python
x = set(("tomato", "brinjal", "potato"))

print(x)

# Note: the set list is unordered, so the result will display the items in a random order.
```

```
{'tomato', 'potato', 'brinjal'}
```

Set Items

Set items are unordered, unchangeable, and do not allow duplicate values.

Unordered

Unordered means that the items in a set do not have a defined order.

Unchangeable

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

Duplicates Not Allowed

Sets cannot have two items with the same value.

In [26]:
```
thisset = {"potato", "brinjal", "potato", "tomato"}
print(thisset)
```

```
{'tomato', 'brinjal', 'potato'}
```

In [27]:
```
thisset = {"potato", "brinjal", "tomato"}
print(len(thisset))
```

```
3
```

Dictionary

Dictionaries are used to store data values in key:value pairs. A dictionary is a collection which is unordered, changeable and does not allow duplicates. Dictionaries are written with curly brackets, and have keys and values:

In [28]:
```
#Create and print a dictionary:
thisdict = {
  "brand": "Ford",
  "day": "snday",
  "date": 3
}
print(thisdict)
```

```
{'brand': 'Ford', 'day': 'snday', 'date': 3}
```

In [29]:

```
#Print the "brand" value of the dictionary:
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])
```

```
Ford
```

Ordered

When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.

Changeable

Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed

Dictionaries cannot have two items with the same key:

In [60]:

```
#Duplicate values will overwrite existing values:
thisdict = {
  "animal": "cat",
  "fruit": "Mango",
  "year": 1964,
  "year": 2020
}
print(thisdict)

```

```
{'animal': 'cat', 'fruit': 'Mango', 'year': 2020}
```

Dictionary Length

To determine how many items a dictionary has, use the len() function:

In [61]:

```python
#Print the number of items in the dictionary:
thisdict = {
  "day": "Sunday",
  "flower": "Rose",
  "year": 1964,
  "year": 2020
}
print(len(thisdict))
```

```
3
```

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

In [62]:

```python
thisdict = {
  "brand": "Ford",
  "electric": False,
  "year": 1964,
  "colors": ["red", "white", "blue"]
}
print(thisdict)
```

```
{'brand': 'Ford', 'electric': False, 'year': 1964, 'colors': ['red', 'white', 'blue']}
```

In [63]:

```python
#Print the data type of a dictionary:
thisdict = {
  "brand": "Ford",
  "fruit": "Mango",
  "date": 19
}
print(type(thisdict))
```

```
<class 'dict'>
```

ARRAYS

Here this shows you how to use LISTS as ARRAYS, however, to work with arrays in Python you will have to import a library, like the NumPy library

In [5]:
```
# Create an array containing fruits names:
fruits = ["Apple", "Banana", "Orange", "mango" ]
print(fruits)
```

```
['Apple', 'Banana', 'Orange', 'mango']
```

Length of an Array Use the len() method to return the length of an array (the number of elements in an array

In [65]:
```
len(fruits)
```

Out[65]: 4

Looping Array Elements You can use the for in loop to loop through all the elements of an array

In [ ]:
```

```

Decision Makings:

If conditions & Loops

Python if Statement

In [1]:
```
# If the number is positive, we print an appropriate message

num = 3
if num > 0:
    print(num, "is a positive number.")
```

```
3 is a positive number.
```

In [2]:
```
num = -1
if num > 0:
    print(num, "is a positive number.")
```

In [3]:
```
a = 7
b = 0
if (a > b):
    print("a is greater than b")
```

```
a is greater than b
```

In [4]:
```
passing_Score = 33
my_Score = 34
if(my_Score >= passing_Score):
    print("Congratulations! You have passed your exam")
```

```
Congratulations! You have passed your exam
```

In [5]:
```
passing_Score = 33
my_Score = int(input("My score is:"))

if(my_Score >= passing_Score):
    print("Congratulations! You have passed your exam")
```

```
My score is:35
Congratulations! You have passed your exam
```

Python if...else Statement

In [6]:
```
num = 5
if(num > 10):
     print("number is greater than 10")
else:
     print("number is less than 10")
```

```
number is less than 10
```

In [7]:

```
passing_Score = 33
my_Score = 67
if(my_Score >= passing_Score):
    print("Congratulations! You passed in the exam")
else:
    print("Sorry! You failed the exam, better luck next time")
```

```
Congratulations! You passed in the exam
```

In [8]:

```
passing_Score = 33
my_Score = int(input("My score is:"))
if(my_Score >= passing_Score):
    print("Congratulations! You passed in the exam")
else:
    print("Sorry! You failed the exam, better luck next time")
```

```
My score is:32
Sorry! You failed the exam, better luck next time
```

elif statements

In [9]:

```
num = 10
if (num == 0):
    print("Number is Zero")

elif (num > 5):
    print("Number is greater than 5")

else:
    print("Number is smaller than 5")
```

```
Number is greater than 5
```

In [10]:

```python
num = -7
if (num > 0):
    print("Number is positive")
elif (num < 0):
    print("Number is negative")
else:
    print("Number is Zero")
```

```
Number is negative
```

Nested if statements

Nested IF functions, meaning one IF function inside of another, allow you to test multiple criteria and increases the number of possible outcomes.

In [11]:

```python
num = 7
if (num != 0):
    if (num > 0):
        print("Number is greater than Zero")
```

```
Number is greater than Zero
```

In [50]:

```python
x = 76
if x > 29:
    print("Above 29")
    if x > 50:
        print("and also above 50")
    else:
        print("but not above 50")
```

```
Above 29
and also above 50
```

Loops:

```
for loop
while loop
```

for loop :

In [14]:

```python
digits = [0, 1, 5]

for i in digits:
    print(i)
else:
    print("No items left.")
```

```
0
1
5
No items left.
```

In [15]:

```python
# Program to find the sum of all numbers stored in a list

# List of numbers
numbers = [6, 5, 3, 8, 4, 2, 5, 4, 11]

# variable to store the sum
sum = 0

# iterate over the list
for val in numbers:
    sum = sum+val

print("The sum is", sum)
```

```
The sum is 48
```

In [16]:

```python
# Program to iterate through a list using indexing
cities = ['ahmedabad', 'mumbai', 'delhi','baroda', 'banglore','pune']

# iterate over the list using index
for i in range(len(cities)):
    print("I like", cities[i])
```

```
I like ahmedabad
I like mumbai
I like delhi
I like baroda
I like banglore
I like pune
```

In [52]:

```python
# program to display student's marks from record
student_name = input("Enter the name:")

marks = {'Jainil': 380, 'Drashti': 382, 'Zeel': 382, 'Prachi':373 ,'Priyanka':342}

for student in marks:
    if student == student_name:
        print(marks[student])
        break
else:
    print('No entry with that name found.')
```

```
Enter the name:Jainil
380
```

In [1]:

```python
list_of_lists = [[1,2,3],[4, 5, 6],[7, 8, 9]]
for x in list_of_lists:
    print(x)
```

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

In [2]:

```python
list_of_lists = [ [1, 2, 3], [4, 5, 6], [7, 8, 9]]
for list in list_of_lists:
    for x in list:
        print(x)
```

```
1
2
3
4
5
6
7
8
9
```

range() Function

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

In [3]:

```python
for x in range(6):
  print(x)
# range(6) is not the values of 0 to 6, but the values 0 to 5.
```

```
0
1
2
3
4
5
```

In [4]:

```
for x in range(2,9):
  print(x)
# range(2, 9), which means values from 2 to 9 (but not including 9)
```

```
2
3
4
5
6
7
8
```

In [5]:

```
# Increment the sequence with 3:
for x in range(2, 30, 3):
  print(x)
```

```
2
5
8
11
14
17
20
23
26
29
```

While loop :

In [6]:

```
x=1
while(x<=5):
    print(x)
    x+=1
```

```
1
2
3
4
5
```

In [7]:

```
# Program to add natural numbers up to sum = 1+2+3+...+n

n = int(input("Enter n: "))

# initialize sum and counter
sum = 0
i = 1

while i <= n:
    sum = sum + i
    i = i+1    # update counter

# print the sum
print("The sum is", sum)
```

```
Enter n: 5
The sum is 15
```

In [20]:

```python
# prints all letters
i = 0
str = 'AppliedMathematics'

while i < len(str):
    print(str[i])
    i += 1
```

```
A
p
p
l
i
e
d
M
a
t
h
e
m
a
t
i
c
s
```

In [1]:

```python
D={'Name':'Dell','Processor':'I5','Ram':'8 GB','Generation':'11th'}
for key, value in D.items():
    print(key,value)

```

```
Name Dell
Processor I5
Ram 8 GB
Generation 11th
```

In [22]:

```python
i=1
while(i<=5):
    print(i)
    i=i+1
else:
    print("The while loop exhausted")
```

```
1
2
3
4
5
The while loop exhausted
```

Nested Loops

A nested loop is a loop inside a loop.

In [9]:

```python
#print each adjective for every fruit:
adj = ["red", "big", "tasty"]
fruits = ["apple", "banana", "cherry"]

for x in adj:
    for y in fruits:
        print(x, y)
```

```
red apple
red banana
red cherry
big apple
big banana
big cherry
tasty apple
tasty banana
tasty cherry
```

break and continue statements

break statement

In [10]:

```
print("The end")

for letter in 'Python':
    if letter == 'o':
        break
    print (letter)
```

```
The end
P
y
t
h
```

In [2]:

```
my_list = ['nike', 'reebok', 'puma', 'adidas', 'campus', 'sketchers']

for i in range(len(my_list)):
    print(my_list[i])
    if my_list[i] == 'campus':
        print('Found the name campus')
        break
```

```
nike
reebok
puma
adidas
campus
Found the name campus
```

In [3]:

```python
i = 0

while True:
    print(my_list[i])
    if (my_list[i] == 'adidas'):
        print('Found the name adidas')
        break
    i += 1
```

```
nike
reebok
puma
adidas
Found the name adidas
```

Continue statement

Continue inside for loop

In [4]:

```python
# If i is equals to 7,continue to next iteration without printing 7.
for i in range(10):
    if i == 7:
        continue
    print(i)
```

```
0
1
2
3
4
5
6
8
9
```

Continue inside while loop

In [5]:

```
i = 0
while i <= 10:
    if i == 7:
        i += 1
        continue
    print(i)
    i += 1
```

```
0
1
2
3
4
5
6
8
9
10
```

Continue inside nested-loop

In [43]:

```
for i in range(4):
    for j in range(4):
        if j==2:
            continue
        print(i,j)
```

```
0 0
0 1
0 3
1 0
1 1
1 3
2 0
2 1
2 3
3 0
3 1
3 3
```

Pass statement :

pass statements are useful in function.

In [13]:
```
a = 33
b = 200

if b > a:
    pass
```

In [14]:
```
for letter in 'Python':
    if letter == 'h':
        pass
```

Functions:

Creating Function

In Python a function is defined using the def keyword

```
def functionname( parameters ):
```

Calling a Function:

To call a function, use the function name followed by parenthesis

In [94]:
```
def my_function():
 print("This is my function")
my_function()
```

```
This is my function
```

Arguments:

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

In [95]:
```
def my_function(fname):
 print(fname + " Patel")

my_function("Richa")
my_function("Kevin")
```

```
Richa Patel
Kevin Patel
```

Number of Arguments:

By default, a function must be called with the correct number of arguments.

Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less

In [96]:
```
#This function expects 2 arguments, and gets 2 arguments

def my_function(fname, lname):
 print(fname + " " + lname)
my_function("Richa", "Patel")
```

```
Richa Patel
```

If we try giving 1 or 3 arguments it will return an error.

In [16]:

```python
def my_function(fname, lname):
    print(fname + " " + lname)
my_function("Richa")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
~\AppData\Local\Temp/ipykernel_13672/732646281.py in <module>
      1 def my_function(fname, lname):
      2     print(fname + " " + lname)
----> 3 my_function("Richa")

TypeError: my_function() missing 1 required positional argument: 'lname'
```

**Arbitrary Arguments, args:*

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a tuple of arguments, and can access the items accordingly:

If the number of arguments is unknown, add a * before the parameter name:

In [18]:

```python
def my_function(*subject):

 print("My fav part in study is " + subject[1]) # try using 0 and 2 instead of 1

my_function("Neurology", "Applications in Mathematics", "Astrophysics")
```

```
My fav part in study is Applications in Mathematics
```

Keyword Arguments:

You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter

In [19]:
```
def my_function(subject1, subject2, subject3):

 print("My fav part in study is " + subject2) # try using other argument

my_function(subject1="Neurology",subject2= "applications in Mathematics", subject3="Astrophysics")
```

```
My fav part in study is applications in Mathematics
```

Arbitrary Keyword Arguments (kwargs)

If you do not know how many keyword arguments that will be passed into your function, add two asterisk: before the parameter name in the function definition.

This way the function will receive a dictionary of arguments, and can access the items accordingly

If the number of keyword arguments is unknown, add a double ** before the parameter name

In [20]:
```
def my_function(**person):
 print("Her last name is " + person["lname"])
my_function(fname = "Richa", lname = "Patel")
```

```
Her last name is Patel
```

In [21]:
```
def my_function(**person):
 print("Her last name is " + person["lname"]+person["fname"])
my_function(fname = "Richa", lname = "Patel")
```

```
Her last name is PatelRicha
```

Default Parameter Value

The following example shows how to use a default parameter value.

If we call the function without argument, it uses the default value:

In [22]:

```
def my_function(country = "Norway"):
 print("I belong to " + country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

```
I belong to Sweden
I belong to India
I belong to Norway
I belong to Brazil
```

Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

In [23]:

```
def my_function(food):
 for x in food:
  print(x)
cusines = ["Italian", "Indian", "Thai"]
snacks= ["beverages", "Burgers"]
my_function(snacks)
```

```
beverages
Burgers
```

Return Values

To let a function return a value, use the return statement

In [24]:

```python
def my_function(x):
 return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

```
15
25
45
```

two values are equal sum will be zero.

In [25]:

```python
def sum(x, y, z):
   if x == y or y == z or x==z:
       sum = 0
   else:
       sum = x + y + z
   return sum
print(sum(2, 1, 2))
print(sum(3, 2, 2))
print(sum(2, 2, 2))
print(sum(1, 2, 3))
```

```
0
0
0
6
```

If the sum is between 15 to 20 it will return 20.

In [26]:

```python
def sum(x, y):
    sum = x + y
    if sum in range(15, 20):
        return 20
    else:
        return sum
print(sum(10, 6))
print(sum(10, 2))
print(sum(10, 12))
```

```
20
12
22
```

Exceptions

Arithmetic Error : Raised when an error occurs in numeric calculations

FloatingPointError : Raised when a floating point calculation fails

ImportError : Raised when an imported module does not exist

IndentationError : Raised when indendation is not correct

Name Error : Raised when a variable does not exist

Overflow Error : Raised when the result of a numeric calculation is too large

Syntax Error : Raised when a syntax error occurs

Tab Error : Raised when indentation consists of tabs or spaces

Value Error : Raised when there is a wrong value in a specified data type

Error not defined

In [27]:

```
def my_function(food):
 for x in food:
  print(x)
fruits = ["apple", "banana", "cherry"]
snacks= ["beverages", "Burgers"]
my_function(Fruits)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
~\AppData\Local\Temp/ipykernel_13672/1144695515.py in <module>
      4 fruits = ["apple", "banana", "cherry"]
      5 snacks= ["beverages", "Burgers"]
----> 6 my_function(Fruits)

NameError: name 'Fruits' is not defined
```

Recursion

Recursion is the process of defining something in terms of itself.

In Python, we know that a function can call other functions. It is even possible for the function to call itself.

These types of construct are termed as recursive functions.

Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

The Python interpreter limits the depths of recursion to help avoid infinite recursions, resulting in stack overflows.

By default, the maximum depth of recursion is 1000. If the limit is crossed, it results in RecursionError.

In [6]:

```python
# Write a Program to find Sum of Natural Numbers using Recursion.

def recur_sum(n):
    if n <= 1:
        return n
    else:
        return n + recur_sum(n-1)
recur_sum(4)
```

Out[6]: 10

change this value for a different result

```
num = 16
if num <0:
print("Enter a positive number")
else:
print("The sum is",recur_sum(num))
```

Think of a recursive version of the function f(n) = 3 * n, i.e. the multiples of 3.

In [6]:

```
def mult(n):
  if n == 1:
    return 3
  else:
    return mult(n-1) + 3
for i in range(1,10):
    print(mult(i))
```

```
3
6
9
12
15
18
21
24
27
```

Find factorial of a number using recursive function.

In [7]:

```
def factorial(x):
    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))
num = 4
print("The factorial of", num, "is", factorial(num))
```

```
The factorial of 4 is 24
```

Global Variables

Variables that are created outside of a function are known as global variables.

Global variables can be used by everyone, both inside of functions and outside.

In [8]:

```python
#Create a variable outside of a function
x = "awesome"
def myfunc():
    print("Python is " + x)

myfunc()
```

```
Python is awesome
```

In [9]:

```python
x = "awesome"
def myfunc():
  #print("Python is " + x)
 myfunc()
print("Python is " + x)
```

```
Python is awesome
```

Write a Program to find Sum of Natural Numbers using Recursion.

In [10]:

```python
def recur_sum(n):
    if n <= 1:
        return n
    else:
        return n + recur_sum(n-1)
recur_sum(4)

# change this value for a different result
num = 16
if num <0:
       print("Enter a negative number")
else:
       print("The sum is",recur_sum(num))
```

```
The sum is 136
```

Local variable

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function.

In [11]:

```python
#Create a variable inside a function
x = "awesome"
def myfunc():
    x = "fantastic"
    print("Python is " + x)
myfunc()
#print("Python is " + x)
```

```
Python is fantastic
```

To change the value of a global variable inside a function, refer to the variable by using the global keyword:

In [12]:

```python
x = "awesome"
def myfunc():
    global x
    x = "fantastic"
myfunc()
print("Python is " + x)
```

```
Python is fantastic
```

Library & Packages

Library is a collection of packages.

For eg: Numpy, pandas, matplotlib, seaborn, scipy, keras

Package is a collection of modules

In [13]:

```python
# import specific attributes from a module without importing the module as a whole.
# importing sqrt() and factorial from the module math

from math import sqrt, factorial

# if we simply do "import math", then math.sqrt(16) and math.factorial() are required.
print(sqrt(16))
print(factorial(6))
```

```
4.0
720
```

Practice Problems

In [14]:

```python
def discriminant():
 x_value = float(input('The x value: '))
 y_value = float(input('The y value: '))
 z_value = float(input('The z value: '))
 discriminant = (y_value**2) - (4*x_value*z_value)
 if discriminant > 0:
   print('Two unique Solutions. Discriminant value is:', discriminant)
 elif discriminant == 0:
   print('One or same Solution. Discriminant value is:', discriminant)
 elif discriminant < 0:
   print('No Real Solutions. Discriminant value is:', discriminant)
discriminant()
```

```
The x value: 5
The y value: 5
The z value: 5
No Real Solutions. Discriminant value is: -75.0
```

Find the Largest Among Three numbers, change the values of num1, num2 and num3 for a different result

In [19]:

```
num1 = 10
num2 = 14
num3 = 12
# uncomment following lines to take three numbers from user
#num1 = float(input("Enter first number: "))
#num2 = float(input("Enter second number: "))
#num3 = float(input("Enter third number: "))
if (num1 >= num2) and (num1 >= num3):
 largest = num1
elif (num2 >= num1) and (num2 >= num3):
 largest = num2
else:
 largest = num3
print("The largest number is", largest)
```

```
The largest number is 14
```

Check length of data and length of data set equal or not

In [22]:

```
def test_distinct(data):
    if len(data) == len(set(data)):
        return True
    else:
        return False;
print(test_distinct([1,5,9,7,9]))
print(test_distinct([2,4,5,7,9])) #try taking different set
```

```
False
True
```

Find given number is odd or even

In [23]:

```python
num = int(input("Enter a number: "))
mod = num % 2
if mod > 0:
 print("This is an odd number.")
else:
 print("This is an even number.")
```

```
Enter a number: 10
This is an even number.
```

Write a Program to Add Two Matrices [nested loop].

Program to add two matrices using nested loop

In [25]:

```python
X = [[12,7,3],
[4 ,5,6],
[7 ,8,9]]
Y = [[5,8,1],
[6,7,3],
[4,5,9]]
result = [[0,0,0],
[0,0,0],
[0,0,0]]
# iterate through rows
for i in range(len(X)):
# iterate through columns
 for j in range(len(X[0])):
        result[i][j] = X[i][j] + Y[i][j]
for r in result:
    print(r)
```

```
[17, 15, 4]
[10, 12, 9]
[11, 13, 18]
```

Write a Program to Multiply Two Matrices [nested comprehension].

Program to multiply two matrices using nested loops.

In [26]:

```
# 3x3 matrix
X = [[12,7,3],
[4 ,5,6],
[7 ,8,9]]
# 3x4 matrix
Y = [[5,8,1,2],
[6,7,3,0],
[4,5,9,1]]
# result is 3x4
result = [[0,0,0,0],
[0,0,0,0],
[0,0,0,0]]
# iterate through rows of X
for i in range(len(X)):
# iterate through columns of Y
 for j in range(len(Y[0])):
# iterate through rows of Y
  for k in range(len(Y)):
      result[i][j] += X[i][k] * Y[k][j]
for r in result:
   print(r)
```

```
[114, 160, 60, 27]
[74, 97, 73, 14]
[119, 157, 112, 23]
```

Write a Program to check if a Number is Positive, Negative or 0 [if else].

Using if...elif...else

In [27]:

```python
num = float(input("Enter a number: "))
if num >0:
    print("Positive number")
elif num == 0:
    print("Zero")
else:
    print("Negative number")
#Here, we have used the if...elif...else statement.
#We can do the same thing using nested if statements as follows.

#Using Nested if
num = float(input("Enter a number: "))
if num >= 0:
    if num == 0:
        print("Zero")
    else:
        print("Positive number")
else:
      print("Negative number")
```

```
Enter a number: 10
Positive number
Enter a number: 5
Positive number
```

Write a Python program to find the median of three values.

In [128]:

```python
a = float(input("Input first number: "))
b = float(input("Input second number: "))
c = float(input("Input third number: "))
if a > b:
    if a < c:
        median = a
    elif b > c:
        median = b
    else:
        median = c
else:
    if a > c:
        median = a
    elif b < c:
        median = b
    else:
        median = c

print("The median is", median)
```

```
Input first number: 5
Input second number: -3
Input third number: 4
The median is 4.0
```

Write a Python program to calculate the harmonic sum of n-1.

In [29]:

```python
def harmonic_sum(n):
    if n < 2:
        return 1
    else:
        return 1 / n + (harmonic_sum(n - 1))

print(harmonic_sum(7))
print(harmonic_sum(4))
```

```
2.5928571428571425
2.083333333333333
```

$$1+\frac{1}{2}+\frac{1}{3}+\frac{1}{4}+\frac{1}{5}+\cdots=\sum_{n=0}^{\infty}\frac{1}{n}$$

Find factorial of a number using recursive function.

In [34]:

```
def fact_loop(num):
    if num < 0:
        return 0
    if num == 0:
        return 1

    factorial = 1
    for i in range(1, num + 1):
        factorial = factorial * i
    return factorial
```

In [32]:

```
def fact_recursion(num):
    if num < 0:
        return 0
    if num == 0:
        return 1
    return num * fact_recursion(num - 1)
```

In [33]:

```
fact_loop(7)
fact_recursion(7)
```

Out[33]: 5040

Program to perform different set operations like in mathematics.

In [35]:
```
# define three sets
E = {0, 2, 4, 6, 8};
N = {1, 2, 3, 4, 5};

# set union
print("Union of E and N is",E | N)

# set intersection
print("Intersection of E and N is",E & N)

# set difference
print("Difference of E and N is",E - N)

# set symmetric difference
print("Symmetric difference of E and N is",E ^ N)
```

```
Union of E and N is {0, 1, 2, 3, 4, 5, 6, 8}
Intersection of E and N is {2, 4}
Difference of E and N is {0, 8, 6}
Symmetric difference of E and N is {0, 1, 3, 5, 6, 8}
```

Write a Python program that will return true if the two given integer values are equal or their sum or difference is 5.

In [36]:
```
def test_number5(x, y):
    if x == y or abs(x-y) == 5 or (x+y) == 5:
        return True
    else:
        return False

print(test_number5(7, 2))
print(test_number5(3, 2))
```

```
True
True
```

In [ ]:

```
1
```